

Take-Home Assignment

Supply Chain Intelligence Mini-Platform

1. Overview

Global shipping moves millions of containers every day. Delays, port congestion, and route disruptions have real business consequences. We want to see how you would build a small, end-to-end slice of a supply-chain intelligence platform.

The assignment is deliberately balanced across three equally-weighted areas:

- **Software engineering** — clean code, REST API design, containerisation, CI/CD, testing (unit and integration), structured logging, and production-readiness thinking.
- **Data engineering** — ingesting and profiling raw data, applying and documenting data quality checks, modelling a curated data product, exposing it via API.
- **AI / ML / GenAI** — a predictive ML model for delay risk, and a lightweight GenAI assistant that uses your data via tool calling.

We care more about clear thinking, sound tradeoffs, and a working thin slice than exhaustive feature coverage or a polished UI.

2. Supplied Data

Three CSV files are included alongside this document:

File	Rows	Description
ports.csv	~25	Reference data — port code, name, country, region, timezone, average congestion score.
shipments.csv	~5,000	Container shipments — booking metadata, planned vs actual departure/arrival, cargo type, weight, status.
port_events.csv	~25,000	Port-level event log — arrivals, departures, delays, inspection notes. Treat this as a streaming-style event feed.

2.1 Embedded Data Quality Problems

The data files contain intentional quality issues. Identifying and handling them is a core part of the assignment. You are not told which rows are affected — finding them is the exercise.

For each issue you find, document it: how you detected it, the scale (how many rows), your decision (drop, fix, flag, quarantine), and your reasoning. Surface the results in a machine-readable format — a JSON report, a quality dashboard endpoint, or a logged summary — your choice.

Important: data quality checks should be a standalone, re-runnable step that does not depend on any ingestion pipeline. They should be runnable against the raw CSV files directly so that quality can be verified before data enters any store.

3. What to Build

There are three parts of roughly equal weight. All three are required. Aim for a working thin slice in each rather than perfection in one.

Part 1 —Engineering & Production Readiness (~4 hrs)

Build the service scaffold that everything else runs on. Even though the domain is data and AI, we want to see solid software engineering fundamentals.

1.1 API Service

1. Build an HTTP API service in any language or framework (FastAPI, Spring Boot, Express, Go, Rails — your choice; state it in the README).
2. Expose at minimum these endpoints:
 1. GET /health — liveness / readiness check, should return service status and version.
 2. GET /shipments/{id} — return full details for one shipment.
 3. GET /shipments — list shipments with at least origin, destination, status, and date-range filters, plus pagination.
 4. GET /routes/{origin}/{destination}/stats — aggregated metrics for a route: average delay hours, on-time rate, shipment count.
 5. POST /predict-delay — call your ML model (built in Part 3) and return a delay-risk prediction.
 6. GET /data-quality/report — return the output of your DQ checks (Part 2) as JSON.
3. Return meaningful HTTP status codes and structured JSON error responses — not just a 500 for everything that goes wrong.
4. Add structured logging (JSON format preferred). Every request should log at minimum: method, path, status code, and latency.

1.2 Testing

Tests are a required deliverable, not optional. We expect two levels:

1. **Unit tests** — cover core business logic (e.g. delay calculation, DQ check functions, ML feature engineering). Mock external dependencies. Tests should run in milliseconds without any infrastructure.
2. **Integration / end-to-end tests** — spin up the service (ideally via docker compose or a test container) and exercise real HTTP requests against real endpoints. Cover at least: a successful shipment lookup, a 404 for an unknown ID, the DQ report endpoint, and the predict-delay endpoint.
3. Tests must pass in CI. If a test is flaky or environment-dependent, document why.

1.3 Containerisation

1. Provide a Dockerfile for the API service.
2. Provide a docker-compose.yml (or equivalent) that brings the full stack up with a single command.
3. A reviewer should be able to go from git clone to a running, testable service in under ten minutes on a standard laptop.

1.4 CI/CD

Aim: set up a CI pipeline (GitHub Actions, GitLab CI, CircleCI, or similar). At minimum it should: install dependencies, run linting, run your full test suite, and build the Docker image.

Option A — Implement it: set up a working CI pipeline and include the green badge in your README. This is the preferred path.

Option B — Document it: include a dedicated section in your README titled “CI/CD Plan” covering: the pipeline stages and what each does, quality gates (what must pass before merge and before deploy), your chosen tooling and why, deployment strategy (how a change gets from a merged PR to a running service),

and rollback mechanism. Be specific — a detailed, well-reasoned plan is scored on the same rubric as a working pipeline.

Both options are equally valid. Choose one and do it well.

1.5 Production Support Runbook

Add a short section to your README titled "Production Support Runbook" covering:

- How to verify the service is healthy after a deployment.
- The two or three most likely failure modes and how you would diagnose them.
- How you would roll back a bad release.
- What you would add next if this were going to production (monitoring, alerting, secrets management, etc.).

Part 2 — Data (~4 hrs)

Turn the raw CSVs into a clean, queryable data product that the API and the ML/GenAI components can rely on.

2.1 Data Quality Checks (standalone — no ingestion required)

Write a data quality module that runs against the raw CSV files before any ingestion takes place. This should be an independent, re-runnable script or command — not a side-effect of loading data.

1. Check for all twelve DQ issues described in Section 2.1, plus any additional issues you find.
2. For each issue, record: check name, rows affected, percentage of total, severity (critical / warning / info), and your recommended action.
3. Produce a machine-readable quality report (JSON, YAML, or similar) as output. This same report is served by the `/data-quality/report` endpoint.
4. The DQ module must be runnable independently: e.g. `python dq_check.py --input ./data/` or an equivalent command.

2.2 Data Store & Tech Choice

Choose a data store that fits the scale and nature of the data. Include a brief section in your README titled "Tech Choices" that explains:

1. Which data store you chose (e.g. DuckDB, SQLite, PostgreSQL, Parquet files, a cloud store) and why.
2. What you would change at 100× the data volume — would you keep the same store or switch, and why?
3. Any other significant tool or library choices (ML framework, LLM provider, test framework, CI platform) and a one-line rationale for each.
4. What you explicitly decided not to do and why — we value scope judgment.

There is no single right answer. We care that you can articulate the tradeoffs clearly.

2.3 Data Pipeline

1. Ingest the cleaned data into your chosen data store using a clearly layered pipeline (e.g. raw → curated → serving, or equivalent stages you define).
2. Derive at least these fields in the curated layer:
 1. `actual_delay_hours` (`actual_arrival - planned_arrival`, null-safe)
 2. `on_time_flag` (true when `delay ≤ 24` hours)
 3. `route_key` (`origin_port + ' → ' + destination_port`)
 4. `transit_days_planned` and `transit_days_actual`
3. Make the pipeline idempotent — running it twice must not corrupt or duplicate data.
4. Document the data model: table names, key fields, how tables relate.

Part 3 — ML + GenAI Assistant (~4 hrs)

3.1 ML: Delay Prediction

1. Train a model that predicts, at booking time, whether a shipment will arrive more than 24 hours late.
2. Use only features that would be available at booking time. Leaking actual_departure or actual_arrival into features disqualifies the model.
3. Document your feature choices and why.
4. Report evaluation metrics honestly — precision, recall, F1, and AUC at minimum. A well-evaluated logistic regression beats a poorly-evaluated gradient boosting model every time.
5. Describe how you would monitor the model in production — what metrics would you track, and what would trigger a retrain?
6. Save the trained model artefact to the repo. The /predict-delay endpoint should load it at startup, not retrain each time.

3.2 GenAI Assistant

Build a lightweight assistant (CLI interface is completely fine — no UI needed) that can answer questions such as:

- *"Which routes have had the highest average delay this quarter?"*
 - *"What is the on-time rate for shipments from Shanghai to Rotterdam?"*
 - *"What is the delay risk for shipment SHP-00421?"*
1. Use any LLM — OpenAI, Anthropic, a local model via Ollama, or similar. State your choice and rough cost per query in the README.
 2. The assistant must use tool calling. Provide at least these tools and let the model decide when to call them:
 1. query_shipments(filters) — reads from your curated data product.
 2. get_route_stats(origin, destination) — returns aggregated delay and on-time metrics.
 3. predict_delay(shipment_id) — calls your ML endpoint.
 3. Add at least one explicit guardrail and document it. Examples: refuse off-domain questions; cap tool calls per session; require the model to cite which tool produced a factual claim.
 4. Log every LLM call: prompt sent, tools called, tool responses, and final model output. Treat this as a production observability requirement.

4. README — Required Sections

Your README.md must include the following named sections:

1. **Getting started** — one command to clone and run the full stack (docker compose up, make run, or equivalent).
2. **Architecture diagram** — hand-drawn, Excalidraw, Mermaid, or any readable format. Show the main components and how they connect.
3. **Tech choices** — see Section 2.2 above. Data store, ML framework, LLM, test framework, CI — brief rationale for each, and what you consciously skipped.
4. **Data quality summary** — a short table of the DQ issues you found, the number of rows affected, and how you handled each.
5. **Production support runbook** — see Section 1.5 above.
6. **CI/CD** — working pipeline badge, or the "what I would have done" section if not implemented.
7. **If I had more time** — a honest paragraph on what you would improve.
8. **Scaling to production** — a few paragraphs on what changes if this handles millions of shipments at 24/7 reliability. What breaks first? What architectural decisions would you revisit?

5. Evaluation Rubric

We share this so you know exactly what we look at.

Area	What we look for
Software engineering	API design, code quality, tests (unit + integration), containerisation, CI/CD (or documented plan), logging, runbook.
Data engineering	DQ checks (standalone + thorough), pipeline layering, data model, idempotency, tech choice justification.
ML + GenAI	Feature engineering without leakage, honest evaluation, tool-call design, guardrails, LLM logging.
Design judgment & write-up	Architecture clarity, tradeoff reasoning, production-scaling thinking, scope judgment.
Reproducibility	Clone → running in < 10 min, tests pass, CI green (or a clear CI plan).

6. Ground Rules

- Any language, framework, or tooling. Justify your choices — that judgment is part of what we assess.
- No UI is required. If you build one that is great.
- A demo walkthrough is not required with submission. We will do a live technical walkthrough in the follow-up interview round.
- Use of AI coding assistants (Copilot, Claude, Cursor, ChatGPT) is expected and welcome. Tell us in the README where AI helped and where you made the key decisions.
- Time-box yourself. If you hit ~14 hours, stop and document what you would do next. Clear scope judgment is a positive signal, not a negative one.
- If something is ambiguous, make a reasonable assumption, note it in the README, and move on. Do not wait for us.
- Share the repo (private) with your hiring contact before the deadline.

7. Submission Checklist

- Private Git repository shared with your hiring contact.
- `docker compose up` (or equivalent) brings the full stack up without extra steps.
- Unit tests pass locally and in CI (or CI plan documented).

- Integration tests exercise real endpoints.
- Standalone DQ check script runs against raw CSV files.
- /data-quality/report endpoint returns the DQ results as JSON.
- README contains all eight sections from Section 4.
- Model artefact is committed — /predict-delay works without retraining.

8. What We Are Not Looking For

- A polished UI or any front-end work — a CLI or Postman collection is completely fine.
- Kubernetes, Terraform, or complex cloud infrastructure.
- A high ML accuracy number — honest evaluation and sound methodology matter far more.
- Every tool from the job description — pick what fits and justify what you skip.
- Perfection in one area at the expense of the other two — we want balance.

Good luck — we look forward to discussing your work in the interview.